Solution Sketches

Problem A – Ananna

Author: Giovanna Kobus Conrado, Brasil

We will maintain an event queue containing pairs of vertices that are connected by palindromic walks. Additionally, we use a boolean matrix to track whether a given pair of vertices has been connected by a palindromic walk at any point (i.e., whether the pair has been added to the queue before).

Initialization

1. The queue is initially populated with:
   - Every pair (u, u) (trivial palindromic walks).
   - Every pair (u, v) where there exists a direct edge from u to v.

Processing the Queue

For each pair (u, v) removed from the queue:

1. Iterate over all incoming edges (a, u, c_1) and outgoing edges (v, b, c_2).
2. If $c_1 = c_2$, then there is a palindromic walk from a to b.
3. If (a, b) has not been previously added to the queue, enqueue it.

Finding the Answer

Once the queue is empty, we output the number of pairs that were added to the queue, excluding pairs of the form (u, u).

Complexity Analysis

Each event (u, v) requires iterating over all incoming edges of u and all outgoing edges of v, which takes time proportional to:

$$\text{INDEG}(u) \times \text{OUTDEG}(v)$$

Summing over all pairs (u, v), the total complexity is:

$$\sum_{u,v} \text{INDEG}(u) \times \text{OUTDEG}(v) = O(M^2)$$

where M is the number of roads in the input. This follows from the fact that the sum of all indegrees and the sum of all outdegrees in the graph are both equal to M, meaning the sum of their products over all pairs is bound by $O(M^2)$.

Note: This approach is analogous to the algorithm for context-free language (CFL) recognition in graphs.

Problem B – Brazilian FootXOR

Author: Jorge Alejandro Pichardo Cabrera, Cuba

The problem can be rephrased as finding a non-empty subset of vectors whose XOR is 0 and whose size is even. To return to the original problem, we can simply assign half of the elements in the subset to team 1 and the other half to team 2.

Ignoring the requirement that the subset size must be even, how can we find a non-empty subset of vectors such that $XOR = 0$? A simple approach is to fix one vector and perform Gaussian elimination for XOR basis on the remaining vectors to check whether there exists a solution to $XOR = 0$ that includes the fixed vector. However, this approach is not efficient enough.

A closer look at Gaussian elimination reveals that every vector reduced to the zero vector during the process is part of at least one valid solution. In other words, for each such vector, there exists a solution that includes it. Thus, we can pick one of these vectors and use Gaussian elimination to find a non-empty subset of vectors whose XOR is 0. If no vector is reduced to the zero vector, then no solution exists, as this implies the vectors are linearly independent, so it is impossible to find a non-empty subset of vectors whose XOR is 0.

Returning to the original problem, how can we ensure that the subset has an even size? The key observation is that if we append 1 to the end of every vector, all solutions for finding a non-empty subset with XOR equal to 0 will be forced to have an even size. This occurs because, for the added dimension to be 0, an even number of vectors is required due to XOR properties. Thus, we can reduce the original problem to the simpler case of finding a non-empty subset whose XOR is 0, which we have discussed above.

Overall complexity: $O(n^3)$ using Gaussian elimination for the XOR basis. The implementation of Gaussian elimination with bitsets optimizes the constant factor enough to comfortably pass within the time limit.

Problem C – Coatless in Yakutsk

Author: Giovanna Kobus Conrado, Brasil

This problem admits multiple simple and efficient solutions, depending on how you interpret the constraints and structure.

You are given a sequence of temperatures over N days and a warm coat that gets dirty after C days of use. The coat starts clean, and once dirty, cannot be worn until it is washed. You may also choose to wash it earlier. On any day the coat is not worn – either because it is being washed or too dirty – you must endure that day's temperature without its protection.

Your goal is to schedule wash days in such a way that the *lowest* temperature on a day you are coatless is as *high* as possible.

Key Insight

Let us consider a threshold temperature T. Suppose you choose to wash the coat on every day with temperature $\geq T$. Then, your coatless days are exactly those days with temperature $\geq T$.

To determine if this strategy is valid for a given T, simply check if any contiguous segment of coatless days has length greater than C. If no such segment exists, T is valid.

Brute Force over Temperature Thresholds

The temperatures are bounded between -50 and 50, so we can afford to check all possible values of T in this range.

For each threshold T, simulate a pass over the temperature array:

- Mark each day as coatless if its temperature is $\geq T$.
- Check for any contiguous segment of coatless days longer than C.

This results in a time complexity of $O(100N)$, which is efficient enough for $N \leq 10^5$.

If temperatures were not bounded, the same logic can be used in a binary search over possible temperature values in $O(N \log N)$.

Sliding Window Approach

We can also frame this as a sliding window problem. Consider sliding a window of size $C + 1$ across the array. Each such segment corresponds to a case where you are forced to be coatless on at least one day.

In each window, compute the **maximum temperature** – this is the worst-case coatless day in that segment. Then, take the **minimum** of these maximums across all windows.

This can be implemented efficiently using:

- A monotonic deque (a.k.a. amortized max queue), or
- Prefix/suffix precomputations for blocks of size $C + 1$.

Component-Based Simulation

Imagine all days initially as coatless. Now, sort the temperatures and "add back" the days from lowest to highest temperature.

As you add back each day, track the sizes of contiguous blocks of coatless days. The moment all such blocks have length at most C, the temperature you just added is your answer.

This approach can be implemented using union-find or segment tracking and runs in $O(N \log N)$ time.

Problem D – Dangerous City

Author: Luan Ícaro Pinto Arcanjo, Brasil

The key observation in this problem is that the danger score of each intersection only depends on the **Minimum Spanning Tree (MST)** of the graph. To see why, note that instead of keeping the danger rating at each intersection, we can transfer it to the edges:

- Define the **weight** of each edge (U, V) as $\max(D_U, D_V)$, where D_U and D_V are the original danger ratings of the intersections.
- With this transformation, the problem reduces to considering the "minimum path risk" in an MST.

Thus, instead of analyzing all paths in the original graph, we can construct the MST and use the resulting **DSU tree** to efficiently compute the required values.

Step 1: Constructing the Minimum Spanning Tree and DSU Tree

We use **Kruskal's algorithm** along with **Disjoint Set Union (DSU)** to construct the MST. During this process, we implicitly build a **DSU tree** as follows:

- Process the edges in increasing order of weight.
- When merging two components in DSU, create a **new virtual node** to represent their union.
- The weight of this new node is set as the **maximum** danger rating among all intersections in its subtree (i.e. the max between the weights of the nodes being merged).
- The new node is linked to the two merged components, forming a **binary tree** structure where:
  - Leaves correspond to the original graph nodes (intersections).
  - Internal nodes represent merge operations.

At the end of this process, the root of the DSU tree represents the entire city, and each leaf corresponds to an intersection from the original graph.

Step 2: Interpreting the DSU Tree to Compute Danger Scores

The DSU tree allows us to efficiently compute the **risk factor** $f(U, V)$, which is the weight of the Lowest Common Ancestor (LCA) of U and V in the tree.

To compute the **danger score** S_U for an intersection U, we observe the path from U to the root of the DSU tree. Using the following formula:

$$S_U = \sum_{i=1}^{k-1} (\text{leaf}_{v_i} - \text{leaf}_{v_{i+1}}) w_{v_i} + w_{l_i}$$

where $v_1, v_2, \ldots, v_k = U$ is the path from the root to U, and leaf_u represents the number of original intersections in the subtree of u.

To efficiently compute S_U for all intersections, we propagate values down the tree using **DFS or BFS** and apply the recurrence:

$$G(U) = \begin{cases} 0, & \text{if } U \text{ is the root} \\ G(\text{parent}_U) + (\text{leaf}_{\text{parent}_U} - \text{leaf}_U) w_{\text{parent}_U}, & \text{otherwise} \end{cases}$$

Since the DSU tree has at most $2N - 1$ nodes, this traversal computes all scores in $O(N)$ time.

Overall complexity: $O(M \log M + N \log N)$.

Problem E – Exciting Business Opportunities

Author: Roberto Sales, Brasil

We have a tree with N nodes (stations) and a list of P operations (proposals), each of which is:

- *Open a business* at some node X, or
- *Sponsor (add a sponsor to)* node X.

A *contiguous range* of these operations (e.g., proposals $i, i+1, \ldots, j$) is called *good* if, *within that range alone*, every opened business is on the path between at least two sponsored nodes, or else is at a sponsored node itself. Formally, a business at node X is valid if:

- Node X itself is sponsored in that range, or
- There exist two *distinct* sponsored nodes s_1 and s_2 whose unique path $s_1 \leftrightarrow s_2$ passes through X.

We want, for each starting index $1 \leq i \leq P$, to find the maximum $j \geq i$ such that the subarray of operations $[i, i+1, \ldots, j]$ is good. Then we output $j - i + 1$ for each i.

Let us assume the tree is rooted at some node a. We define $F_a(b)$ as follows:

- If $b = a$, then $F_a(b) = a$.
- Otherwise, let $F_a(b)$ be the *direct child* of a in the rooted tree on the unique path from a to b.

With this definition, a business proposal B_i on node a is valid if:

- There is a sponsor proposal in node a, or
- There exist two sponsor proposals in nodes s_1 and s_2 such that $F_a(s_1) \neq F_a(s_2)$ (meaning s_1 and s_2 lie in different child-subtrees or sides of a, so the path $s_1 \leftrightarrow s_2$ goes through a).

For a business proposal B_i at node a, we look for *sponsor proposals* on both sides in the *proposal list* (indices less than i, and indices greater than i). Concretely:

- **Left side (indices $< i$):**
  - $L1_i$: The rightmost sponsor proposal (in index order) on some node b_l with index $< i$.
  - $L2_i$: Another rightmost sponsor proposal on some node c_l with index $< i$, subject to $F_a(b_l) \neq F_a(c_l)$ and $F_a(b_l) \neq a$.
- **Right side (indices $> i$):**
  - $R1_i$: The leftmost sponsor proposal (in index order) on some node b_r with index $> i$.
  - $R2_i$: Another leftmost sponsor proposal on some node c_r with index $> i$, subject to $F_a(b_r) \neq F_a(c_r)$ and $F_a(b_r) \neq a$.

These four values $(L1_i, L2_i, R1_i, R2_i)$ capture the critical sponsors that might combine to validate B_i.

By analyzing these sponsor proposals, we observe that a business proposal B_i on a is valid if at least one of the following intervals (in terms of proposal indices) exists and is valid:

- $[L1_i, B_i]$: Exists if $L1_i$ exists and $F_a(b_l) = a$.
- $[B_i, R1_i]$: Exists if $R1_i$ exists and $F_a(b_r) = a$.
- $[L1_i, R1_i]$: Exists if both $L1_i$ and $R1_i$ exist, and $F_a(b_l) \neq a$, $F_a(b_r) \neq a$, $F_a(b_l) \neq F_a(b_r)$.
- $[L2_i, B_i]$: Exists if $L2_i$ exists.
- $[B_i, R2_i]$: Exists if $R2_i$ exists.

In practice, these intervals represent the subranges of proposals where B_i is guaranteed coverage from the required sponsors. With these definitions, we need to determine how to compute $F_a(b), L1_i, L2_i, R1_i$, and $R2_i$, and how to use these intervals to solve the problem.

Computing $F_a(b)$

One way to compute $F_a(b)$ is via an Euler tour (on any initial root) and subtree checks:

1. Perform an Euler tour from a convenient root to record $t_{in}(v)$ and $t_{out}(v)$, and subtree sizes.
2. To handle "root at a", check if b is inside a's DFS interval:
   - If not, treat $F_a(b)$ as the "parent side" of a,
   - If yes, let ch be the child of a whose interval covers b; then $F_a(b) = ch$. For this we need to find the child ch of a such that $t_{in}(ch) \leq t_{in}(b) \leq t_{out}(ch)$. To efficiently determine this, we can store, for each a, the entry and exit times of its children in a set and use a lower bound or binary search on this set.

Computing $L1_i, L2_i, R1_i, R2_i$

Finding $L1_i$ simply means the nearest sponsor to the left of i (largest index $< i$). Once b_l is identified, we can check $F_a(b_l) \neq a$ to see if we need $L2_i$.

Now, assume that $F_a(b_l) \neq a$ and focus on computing $L2_i$.

To compute $L2_i$, we construct a segment tree over the Euler tour of the tree, initializing every element to -1. Using this segment tree, we iterate over the business proposals from left to right, performing the following steps:

- If proposal i is a sponsor proposal on the node a, update the position $t_in(a)$ in the segment tree with the value i.
- If proposal i is a business proposal on the node a, we can analyze node b_l in the $L1_i$ proposal:
  - If $F_a(b_l)$ is the parent of a, then $L2_i$ is given by querying the interval $[t_{in}(a), t_{out}(a)]$. If this query returns -1, then $L2_i$ does not exist.
  - If $F_a(b_l)$ is the child ch of a, then $L2_i$ is the maximum of the queries on the intervals $[1, t_{in}(ch) - 1]$ and $[t_{out}(ch) + 1, N]$. If this value is -1, then $L2_i$ does not exist.

Similarly, we can do a separate pass from right to left to identify $R1_i$ and $R2_i$ (the earliest sponsors with index $> i$) that can validate B_i.

Using Interval Information to Solve the Problem

Once each business knows which sponsor intervals can validate it, the tree structure is no longer needed. We solve an *interval processing* problem:

- A proposal is **sponsor** if it adds a sponsor.
- A proposal is **business** if at least one of its intervals is valid in the chosen subarray.

We consider two main approaches to find the largest valid subarray starting at each index:

1. *Sweep-line with a segment tree*, adding sponsors/businesses as we move from left to right and tracking validity.
2. *Divide-and-conquer*, marking invalid businesses and removing intervals whenever we detect a subarray that fails.

Divide-and-Conquer Sketch

- If a business is invalid (no valid intervals remain), we find it and split the subrange around it.
- If all businesses in $[l, r]$ are valid, then the entire segment is good; record $answer_l = r$ and move on.

This runs in $O(P \log P)$, and combined with Euler tour plus sponsor lookups in $O((P + N) \log N)$, we get a total complexity of $O(P \log P + (P + N) \log N)$.

In short... By identifying $L1_i, L2_i$ (on the left) and $R1_i, R2_i$ (on the right) for each business in terms of proposal indices, we can ensure all businesses meet the path requirement. Then, either via sweep-line or divide-and-conquer, we compute for each starting proposal i the maximum j such that $[i, \ldots, j]$ remains good.

Problem F – Festival Signs

Author: Agustín Santiago Gutiérrez, Argentina

This problem can be solved by using the "divide and conquer over queries + rollback" approach typically used for dynamic connectivity, but over a segment tree data structure instead of DSU.

Indeed, it is possible to create an online solution with segment tree to the problem without removal events. Each segment tree operation only modifies $O(\lg N)$ values, and thus can also be roll-backed in $O(\lg N)$ time by storing a list of modifications. Alternatively, a persistent segment tree can be used for this, since persistence is even stronger than rollback-capability. Using the previously mentioned pattern with this solution, one can get an $O(N \lg^2 N)$ offline solution to the whole problem (treat adding a sign / removing a sign analogously to adding an edge / removing an edge in the dynamic connectivity algorithm).

To solve the only-additions version with segment tree, the minimum value can be stored as segment tree data, and also a lazy-update value indicating that "a sign of height H has to be added in this range" can be kept. These things compose nicely and thus can be used with segtree lazy propagation: i.e. adding signs of heights H_1 and then H_2 is the same as adding height $\max(H_1, H_2)$, and when adding a sign of height H to a range, previous minimum M over that range gets updated to $\max(M, H)$.

A constant-factor optimization (particularly memory-wise) that can be performed in this problem is based on noticing that we can store just the minimum and not the lazy update data: we can assume at all times that there is still a propagation-pending operation of a sign having the minimum height in the range, as that will not change the answer, but avoids explicitly storing and cleaning a lazy-propagation data field.

Problem G – Game of Pieces

Author: Rafael Grandsire, Brasil

To determine whether a piece makes the game unsafe, it is sufficient to check the heights of the columns covered by the piece and ensure they are all the same.

- If the heights are equal before placing the piece, the board remains safe.
- If there is any height difference among the columns covered by the piece, then dropping the piece will create an empty cell with a filled cell above it, making the game unsafe.

This observation significantly reduces the complexity of checking for unsafe states. Instead of scanning the entire board, we only need to inspect the columns affected by each piece.

Approach 1: Ad Hoc Implementation (Using a Set of Intervals)

Since we only need to check for height uniformity, we can maintain a *set of intervals* where each interval represents a contiguous block of columns that share the same height.

Operations on the set:

- **Finding the height of an interval** can be done in $O(\log s)$, where s is the number of intervals.
- **Updating a range** requires:
  - Identifying the affected interval.
  - Removing the old interval and inserting at most **three** new ones (left part, updated part, right part).
  - Merging adjacent intervals with the same height to maintain maximality.
- **Incrementing a single position** is similar but affects only $[p, p]$.

Implementation Details

We maintain a **set of triples** $\{L, R, x\}$, where:

- L, R denote the range of columns covered by the interval.
- x represents the height of that segment.

Operations

1. **Checking for equality**
   - Given a range $[l, r]$, we check if it falls within a single interval.
   - This can be done in $O(\log s)$.
2. **Updating a range**
   - Identify the interval $\{L, R, x\}$ containing $[l, r]$.
   - Remove it and insert:
     - $\{L, l-1, x\}$ (left portion, if non-empty)
     - $\{l, r, x+1\}$ (the updated portion)
     - $\{r+1, R, x\}$ (right portion, if non-empty)
   - If adjacent segments now have the same value, **merge them** to maintain maximality.
3. **Incrementing a single column**
   - Similar to updating a range, but affects only $[p, p]$.

Complexity Analysis

Each operation involves:

- **Finding the relevant range** in $O(\log n)$.
- **Updating at most three intervals** and merging in $O(\log n)$.

Thus, the overall complexity is $\mathbf{O(n \log n)}$ for both time and memory.

Approach 2: Sparse Segment Tree

Alternatively a **Segment Tree** can be used to maintain column height information.

We build a Segment Tree where each node stores:

- **Minimum height** in the range.
- **Maximum height** in the range.

This allows us to:

1. **Check for uniformity** in $O(\log n)$ by comparing min and max.
2. **Increment a range** in $O(\log n)$, updating affected nodes.
3. **Increment a single column** in $O(\log n)$.

Implementation Strategy

- The tree supports **lazy propagation**, ensuring updates are efficient.
- Instead of explicitly storing all 10^{18} columns, we use a **Sparse Segment Tree** that dynamically creates nodes as needed.

Complexity Analysis

- **Time Complexity**: $O(n \log n)$ for both updates and queries.
- **Memory Complexity**: $O(n \log n)$ due to the sparse nature of the tree.

Problem H – Horrible Restaurants

Author: Agustín Santiago Gutiérrez, Argentina

Horrible Restaurants is a more complicated version of `https://codeforces.com/problemset/problem/436/E`. That problem is basically the 2-star version, and also the target number of stars is fixed instead of having to output all possibilities. There might also be some resemblance with `https://cses.fi/problemset/task/2426`.

These two previous problems can both be solved by sorting and trying all the "cut points", as explained in the codeforces tutorial. After a careful sort, a solution can be found where everything has 0 or 1 stars on one side, and everything has 1 or 2 stars on the other side, so by iteratively moving a cut point and efficiently answering a query at each step, everything can be checked in $O(N \lg N)$ time. Codeforces mentions a solution using BIT or Cartesian Tree, but a data structure made from two C++ sets and sum-accumulators is also enough.

Horrible Restaurants has two big extra difficulties. The first extra difficulty is the fact that the target number of stars is not fixed, but we have to efficiently compute the answer for all possible values of total number of stars. In the cses problem, we can actually notice that it can be modeled by min-cost max-flow, and then apply the known greedy solution for that specific graph to iteratively build optimal solutions with flow value 1, flow value 2, etc. That would give a practical greedy algorithm to compute the solution for all possible values of $a + b$.

However, in the codeforces star problem there is no min-cost max-flow model. Such an approach (at least when done in some direct way, so that the flow maps to the total number of stars) is

impossible because min-cost max-flow is a linear program and thus the total cost as a function of the number of stars should be convex, which can be seen to not be the case at all by checking some examples. Luckily, quite a similar incremental greedy approach can be shown to work specifically for the star problem (in the case of only 2 stars) even if it does not correspond to a flow algorithm. We start with the minimum number of stars, that is, every restaurant has a 0 stars review. Then iteratively at each step, we will make one "modification operation" to the current optimal solution for k total stars, to get an optimal solution for $k+1$ stars. Allowed modification operations are either increasing the number of stars of a single restaurant by one star, or else choosing one restaurant at 0 stars to increase up to 2 stars, while decrementing 1 star to another restaurant (either changing 2 stars to 1 star, or changing 1 star to 0 stars). Of course, from all possible modification operations at each step, we must be able to efficiently select and perform the one that will achieve the lowest possible final cost. This can be done with a few C++ sets and some care.

If we think of our solution as a vector of numbers indicating the target number of stars at each restaurant, then a modification operation is either choosing one position to do $+1$, or choosing a position to do $+2$ and another one to do -1.

It can be proved that from **any** non saturated (that is, such that some restaurant has less than maximum stars) optimal solution with k total stars, there exists some modification operation that achieves an optimal solution for $k+1$ total stars. Proof: let A be a vector corresponding to an optimal k star solution. Let B be a vector corresponding to an optimal $k+1$ star solution, **as similar** to A as possible (that is, differing in as few positions as possible). Then the total sum of values of vector $B-A$ is 1. Furthermore, if a subset of positions of $B-A$ adds up to a value of 0, they must all be 0: otherwise, either B could be changed without affecting total cost in a way to make it more similar to A, or else one of A or B would not be optimal for its corresponding total number of stars.

For the 2 stars version, this restricts the values in $B-A$ a lot: there cannot be a -2 at all, and there cannot be both a -1 and a $+1$. The only remaining possibilities happen to be the described modification operations of $+1$ or $+2, -1$, thus completing the proof for the 2 stars version.

The second extra difficulty in Horrible Restaurants is that we now have up to 3 stars instead of up to 2 stars. Solutions based on sorting and iterating seem to fail now, as there would be two cut points and testing all pairs gives N^2 time at least. Fortunately, all of the previous ideas for a key "modification operation" can be extended to the case of 3 stars (and any other number of stars, but the number of possible "modification operations" is growing a lot with the maximum number of stars). In this case, atomic operations are found by thinking what possible vectors $B-A$ we can have as in the previous proof, such that total sum is 1, values are in $[-3, 3]$ and no non-zero subset adds up to 0. The result is exactly these five operations:

- -3 to some restaurant A, $+2$ to some restaurant B, and $+2$ to some restaurant C.
- -1 to some restaurant A, -1 to some restaurant B, and $+3$ to some restaurant C.
- -2 to some restaurant A, and $+3$ to some restaurant B
- -1 to some restaurant A, and $+2$ to some restaurant B
- $+1$ to some restaurant A

For each of these five options, it is possible to efficiently find the best way to do it at every step, by keeping $O(1)$ sets carefully. More specifically, set S_d can keep all restaurants such that a $+d$ modification is possible, ordered by the net cost of doing such a change in the number of stars ($-3 \leq d \leq 3$). Slight care must be taken when handling the $+2, -1$ case, as a single

restaurant could be the best -1 option and also the best $+2$ option. These can be solved by looking at the best two elements in that case, in any of various different ways. Segment Trees can also be used instead of sets.

Problem I – Infinite Arrays

Author: Luis Santiago Re, Argentina

For simplicity, the complexity analysis is done using N as "input size".

Let's analyze a single query. Let's call Z to the longest common substring between P^∞ and A^∞ (then the answer to the query is $|Z|$).

Lemma 1: ${A^\infty}_i = {A^\infty}_j \longleftarrow |i-j|$ is divisible by K.

Lemma 2: ${P^\infty}_i = {P^\infty}_j \longleftrightarrow |i-j|$ is divisible by $|P|$. Because the elements of P are pairwise distinct.

Let's assume that $|Z| \geq K+1$ and analyze this case:

(a) From lemma 1, $Z_1 = Z_{K+1}$
(b) From lemma 2 and (a), K is divisble by $|P|$.
(c) From both lemmas and (b), P is a period of A; and then $|Z| = \infty$

So now we know that $|Z| \leq K$ or $|Z| = \infty$. And this means that the answer can be found in A^2 as all subarrays of length at most K of A^∞ are present in A^2.

Then, computing the following values is enough to find the answer: for each position i in A^2, considered as right end of a subarray, compute the left most valid beginning L_i.

Let's call $PosOf$ to an array where $PosOf_{P_i} = i$, i.e. $PosOf_v =$ position in P where the value v is located.

Then, L_i can be computed as follows (asume 0-indexing is used for simplicity):

- If $A_i \notin P$, then $L_i = i+1$ (i.e. no valid subarray ends at A_i)
- If $PosOf_{A_i} = (PosOf_{A_{i-1}} + 1)\%|P|$, then $L_i = L_{i-1}$. Simply extend the best subarray that ends at A_{i-1}.
- If $PosOf_{A_i} \neq (PosOf_{A_{i-1}} + 1)\%|P|$, then $L_i = i$. Because the value A_i is never to the right of value A_{i-1} in P^∞.

So the queries can be solved in $O(\sum K)$.

Note that the actual positions don't matter in $PosOf$, having a pointer to the value and being able to find the "next" and "prev" elements is enough.

So, all that remains is to handle insert/delete, and keep P and $PosOf$ updated. This can be done with a linked list or an implicit treap, making the final complexity $O(N)$ or $O(NlogN)$ respectively.

Other slower solutions like $O(Nlog^2N)$ treap + hash + binary search, or $O(NsqrtN)$ sqrt-decomposition, are very unlikely to be fast enough to pass.

Problem J – Just Look Up

Author: Giovanna Kobus Conrado, Brasil

This problem can be interpreted geometrically as follows: given N points on the surface of a sphere (provided via 3D coordinates), find the largest possible empty spherical cap that does not contain any of the points. In other words, we are looking for the largest possible cone (with its apex at the origin) that avoids all input points.

Geometric Formulation

Let each input point be a unit vector on the sphere. We want to find a direction vector v on the unit sphere such that the maximum inner product between v and any input point is minimized:

$$\min_{v \in \mathbb{S}^2} \max_{i=1}^{N} \langle v, p_i \rangle$$

Let p be this maximum inner product. The angle θ between v and the closest point then satisfies $\cos\theta = p$, so the answer is simply $\theta = \arccos(p)$. If $p < 0$, this means the cap contains a full half-space and the answer is $90°$.

For $N \leq 3$, it is always possible to find an empty half-space, so the answer is always $90°$.

Naive $O(N^4)$ Approach

An important geometric fact is that the largest empty circle must be defined by at least three points on its boundary. This leads to a brute-force solution:

- Iterate over all triples of points.
- For each triple, compute the normal to the plane they define.
- This normal (and its opposite) defines a potential direction vector v.
- For each such v, compute the maximum inner product with all input points.
- Keep the v that minimizes this maximum.

This results in an $O(N^4)$ solution. It is correct, but too slow for the input limits.

Optimized $O(N^3)$ Approach

We can improve the brute-force by observing that the optimal direction vector v is often perpendicular to the plane defined by two or three input points. A more efficient approach is:

- Iterate over all pairs of points (A, B).
- For each such pair, iterate through the remaining points to find a circle that:
  - Has A and B on its boundary,
  - Contains no other point inside.
- For each direction (one on each side of the plane through A and B), test the maximum inner product and keep the best.

This runs in $O(N^3)$ and is efficient enough given the problem constraints.

Convex Hull Based $O(N^2)$ Solution

A geometric insight is that any set of N points on a unit sphere forms the vertex set of a convex polyhedron. The optimal direction vector v must be perpendicular to one of the faces of this convex hull.

- Compute the 3D convex hull of the N input points.
- The number of faces of a convex polyhedron with N vertices is $O(N)$.

- For each face, compute the normal vector (and its opposite), and evaluate the corresponding maximum inner product.

This yields an $O(N^2)$ to $O(N^3)$ solution depending on the convex hull implementation. Given the low constant factors and small limits, using a standard 3D convex hull algorithm is sufficient.

Alternative Perspective-Based Solution

One team proposed a clever alternative using projection:

- Fix a point A and rotate the system so that A becomes $(1, 0, 0)$.
- Project all other points onto the plane $x = 0$ via **perspective projection** from A.
- Under this projection, circles passing through A become straight lines.
- Then, the problem reduces to checking whether a line segment (e.g., BC) is an edge of the convex hull of the projected points.

This reduces the problem to 2D convex hull checks in a transformed space. The projected coordinates of a point (x, y, z) become:

$$(0, \frac{y}{1-x}, \frac{z}{1-x})$$

This is effectively a stereographic-like projection with the viewpoint at $(1, 0, 0)$.

Problem K – Keep Fighting

Author: Luan Ícaro Pinto Arcanjo, Brasil

Bob cannot defeat the monster if there are no attack cards or if his power remains 0 throughout the game. From this point onward, we will assume that Bob can defeat the monster.

The first key observation in solving this problem is that if Bob cannot defeat the monster before the deck resets, it is optimal to play the add cards first, then the multiply cards, and finally the attack cards. This can be proven by an exchange argument.

From this key observation, we can also infer that if Bob can defeat the monster before the deck resets, we can discard the multiply-by-1 cards and then brute-force the number of add, multiply, and attack cards to play. In this process, it remains optimal to play the add cards first, followed by the multiply cards, and finally the attack cards.

The next key observation is that the process of:

- If Bob cannot defeat the monster before the deck resets, he simply plays the cards in optimal order.
- If Bob can defeat the monster before the deck resets, brute force the number of add, multiply, and attack cards to play, prioritizing the highest value add and multiply cards first.

is in fact fast enough, except in the edge case where Bob's power does not increase, which we will discuss later.

If Bob's power increases, the number of deck resets is $O(\sqrt{h})$. This happens because the attack cards deal strictly more damage than before the last reset. In the worst case, the damage before the first reset is 1, before the second reset is $1+2$, etc. Thus, after m resets, Bob will deal at least $\sum_1^m i$ damage, and the minimum m such that $\sum_1^m i \geq h$ is in the order of $O(\sqrt{h})$.

Checking whether the monster can be defeated before the deck resets can be done in $O(n)$, but the problem constraints allow us to brute-force first to determine if it is possible.

The final complexity for the case where Bob's power increases is $O(n \times \sqrt{h} + n^3)$:

- $O(n \times \sqrt{h})$ to simulate the process until Bob can defeat the monster without resetting the deck again.
- $O(n^3)$ to brute-force the number of add, multiply, and attack cards needed to defeat the monster before another deck is reset.

Bob's power does not increase

If Bob's power does not increase, then his deck may contain only two types of cards:

- Attack cards (**!**), let A be the number of attack cards.
- Multiply-by-1 (*** 1**), let M_1 be the number of multiply-by-1 cards.

Since Bob's power remains constant, the number of attack cards required to defeat the monster is:

$$need = \lceil \frac{h}{p} \rceil$$

Since the deck resets after all cards are played, the number of times the deck will be reset is:

$$r = \lceil \frac{need}{A} \rceil - 1$$

because Bob needs to play at least $need$ attack cards, and each full-deck cycle allows him to play$A$ attack cards.

Thus, the total number of cards played is:

$$answer = need + r \times M_1$$

because each deck reset introduces M_1 additional multiply-by-1 cards that must be played before Bob can attack again.

Problem L – LED Counter

Author: Alejandro Strejilevich de Loma, Argentina

Each of the N positions of the counter can be determined independently of the other positions. So we will restrict our attention to the case $N = 1$.

Algorithms that check each digit

The general idea of this type of algorithms is to check whether each digit $0, 1, \ldots, 9$ can be described as the input string indicates. If a single digit can be described that way, output that digit; otherwise output "*". As we will see, the cost of any of these solutions is $O(1)$ (for each position of the counter), although the specific constants are different depending on how each digit is checked.

- Check each digit by brute force:

  For deciding whether a digit can be described as the input string indicates, determine whether each LED of the string is compatible with the corresponding LED of the digit. LEDs "`+`" and "`-`" in the string are always compatible, while LEDs "`G`" and "`g`" must be coincident with those that represent the digit. This solutions checks each LED of each digit against the string, so the constant is proportional to 10×7.

- Check each digit using regular expressions:

  At the beginning of the program, build a regular expression for each digit, representing the possible ways in which the digit can be described by the input string. For doing so, note that a turned on LED in the digit can be described as "`G`", "`+`" or "`-`", while a turned off LED can be described as "`g`", "`+`" or "`-`". For instance, since the digit 0 is represented as "`GGGgGGG`", its associated regular expression is

  ```
  (G|\+|-)(G|\+|-)(G|\+|-)(g|\+|-)(G|\+|-)(G|\+|-)(G|\+|-)
  ```

  For deciding whether a digit can be described as the string indicates, just check whether the string matches the regular expression associated to the digit. This solution tries to match the string (of length 7) with simple regular expressions associated to the digits, so the constant is proportional to 10×7 . However, in practice this solutions is slower than the previous one because of the overhead involved in using regular expressions.

- Check each digit using bitmasks:

  It what follows, $\vee$ means bitwise OR, $\wedge$ means bitwise AND, while $-$ means bitwise NOT.

  At the beginning of the program, define a bitmask B_i indicating the LEDs that should be turned on while displaying digit i in an error-free device. For instance, $B_0 = 1110111_2$.

  Split the input string into three bitmasks of length 7: B^+ indicates the always-on LEDs, B^- indicates the always-off LEDs, while B indicates the LEDs that are turned on. Note that $B^+ \wedge B^-$ is null (no LED is both always-on and always-off).

  For deciding whether digit i can be described as the string indicates, compute

  $$D_i = \left[B_i \vee B^+\right] \wedge \left(-B^-\right) = \left[B_i \wedge \left(-B^-\right)\right] \vee \left[B^+ \wedge \left(-B^-\right)\right] = \left[B_i \wedge \left(-B^-\right)\right] \vee B^+ \ ,$$

  which indicates the LEDs that should be turned on while displaying digit i in the available device. The digit can be described as the string indicates if and only if $B = D_i$.

  This solution builds the bitmasks B^+, B^- and B, and then for each digit i computes D_i and compares it against B. Thus, the constant is proportional to $7 + 10$.

Algorithm that builds a set of candidates

Instead of checking each digit, a possible approach it to consider a set of digits that can be described as the input string indicates. Initially, the set contains every digit $0, 1, \ldots, 9$. After that, for each LED in the string remove from the set those digits that are not compatible with the LED. Finally, if the set contains a single digit, output that digit; otherwise output "`*`". The cost depends on how the set is implemented. Using bitmasks the cost is proportional to $7 + 10$ for each position of the counter. The Python code below shows this approach.

```python
#!/usr/bin/env python3

def BuscarCandidatos(Display):
    CandidatosConLed=[0b1101110001, 0b0101000101, 0b1111101101, 0b1101111100,
                      0b1101101101, 0b1110011111, 0b1111111011]
    Candidatos=0b1111111111
    for i in range(7):
        if Display[i]=='G': Candidatos&=CandidatosConLed[i]
        elif Display[i]=='g': Candidatos&=~CandidatosConLed[i]
#   if Candidatos==0: return -2               # none (does not occur according to statement)
    if Candidatos & (Candidatos-1): return -1 # many (not a power of 2)
    Digito=0
    while Candidatos>1:
         Digito+=1
         Candidatos>>=1
    return Digito                             # unique

Cero=ord('0')
Contador=''
N=int(input())
for _ in range(N):
    Candidato=BuscarCandidatos(input())
    Contador+=('*' if Candidato<0 else chr(Cero+Candidato))
print(Contador)
```